

1. Executive Summary

AgriFlow is a simplified, offline-first farm management PWA designed to replace paper notebooks for family farms. It focuses on geospatial navigation (fields on a map) and high-speed financial/agronomic data entry. The core technical differentiator is its ability to handle complex field-level P&L calculations while functioning entirely without cellular service.

2. User Personas

Persona A: "Harvest Harry" (Primary)

- **Profile:** 58-year-old owner/operator. Uses a mid-range Android phone. Eyesight is average; hands are often gloved or dirty.
- **Tech Literacy:** Low. Gets frustrated by small buttons, loading spinners, and login screens.
- **Behavior:** Checks the app while in the tractor cab or at the gas station.
- **Key Constraint:** "If I have to tap more than 3 times to enter a fuel receipt, I'm going back to my notebook."

3. Functional Requirements

3.1 Geospatial Dashboard (Home)

- **Map View:**
 - Implement **Mapbox GL JS**.
 - Render saved fields as **Polygons** overlaying the satellite layer.
 - **Color Coding:** Fields color-coded by crop type (e.g., Corn = Yellow, Soy = Green) or Profitability (Green = Positive, Red = Negative).
 - **Interaction:** Tapping a polygon opens the **Field Detail View**.
- **Field Creation:**
 - "Draw on Map" feature to define field boundaries (GeoJSON).
 - Auto-calculate "Acres" based on polygon area, but acreage can also be manually entered if they have the information.

3.2 Offline First & Sync (Critical)

- **Local Storage:** The app must load the full state from IndexedDB (recommend usage of **Dexie.js** or **TanStack Query with Persisters**) immediately on launch.
- **Queue System:** Modifications made offline must be added to a "Sync Queue."
- **Reconnection:** When navigator.onLine is true:
 1. Process the Sync Queue (POST/PUT to Supabase).
 2. Pull latest data from Supabase (Conflict Resolution: Last-Write-Wins based on updated_at timestamp).
 3. Update Local Storage.

3.3 Field-Level Financials & Data Entry

- **The "Log" Action:** A Floating Action Button (FAB) (+) on the dashboard and field view to add a record.
- **Transaction Types:**
 1. **Income:** Subsidies, Insurance, Commodity Sales.
 2. **Expense:** Inputs, Machine Hire, Taxes, Interest.

3.4 Data Visualization

- **Charts:** Use **Recharts** for lightweight mobile rendering.
- **Views:**
 - *Field View:* Stacked Bar Chart (Expenses broken down by category vs. Total Income).
 - *Global View:* Net Income per Acre (Leaderboard of most profitable fields).

3.5 Farm Configuration & Entity Logic (The "Sharecropping" Constraint)

- **Problem:** The user may not own 100% of every crop.
- **Requirement:**
 - **Season Setup:** When starting a new season (e.g., "2025 Corn") for a specific field, the user must define:
 - **Operator Share %:** (e.g., 100% for owned land, 66% for sharecropped).
 - **Landlord Name:** (Optional text field).

- **Impact:** All "Net Income" calculations on the dashboard must apply this percentage multiplier to Revenue (and potentially shared expenses).

3.6 Advanced Financial Recording

A. Revenue Streams (Detailed)

1. Quality Metrics (The "Dockage" Check):

- Input fields for **Moisture %**, **Test Weight**, and **Protein**.
- *Why?* High moisture reduces net weight/pay. This data is critical for the "Per Acre" audit.

2. Government Payments:

- **Mus be able record not just that there were government payments but what kind of payments they were.**
 - **Program Selector:** Must allow selecting specific subsidy programs (eg. **PLC, ARC, CFAP, ERP, MFP**) this should be a dropdown field, but they should have the ability to add more to it when new programs arise.
 - *Why?* Farmers need to know if they are profitable from *farming* vs. *subsidies*.

B. Expense Recording (Vendor & Product Context)

1. Vendor Tracking:

- Every expense must have a "Payee/Vendor" field (e.g., "MKC", "CPS", "Sprayer Co").
- *UI Requirement:* Auto-complete based on previous entries.

2. Chemical/Fertilizer Specifics:

- **Product Name:** Instead of just "Herbicide," have the ability to capture specific names like "Liberty 280", "Zidua", "Finesse".
- **Application Stage:** Dropdown to indicate *timing* (at what stage of crop growth was it applied):
 - *Options:* Pre-Plant, Starter, Top Dress, Spring Weed, Fall Weed, Desiccant.

- **Weather Context:** A generic weather_notes field (e.g., "Applied 12hrs before rain").

3.7 Nutrient Loading (The "Actual N" Calculation)

- **Requirement:**
 - When logging fertilizer, providing a "Calculator" helper:
 - *Input:* Total Lbs of Product (e.g., Urea 46-0-0).
 - *Input:* Product N-P-K rating (e.g., 46-0-0).
 - *Output:* Auto-calculate **Actual Lbs of N per Acre**.
 - *Data Storage:* Store both the *Cost* and the *Actual N* value.

4. API & Sync Strategy

4.1 Technology Choice: TanStack Query + Supabase

We will not build a custom REST API. We will use the **Supabase JavaScript Client** directly in the React frontend, wrapped in **TanStack Query** for caching and state management.

4.2 Offline "Optimistic" Workflow

1. **State:** Use zustand or TanStack Query with persist-client to save the mutation queue to localStorage.
2. **Image Uploads:**
 - *Offline:* Store image as Base64 string in IndexedDB.
 - *Offline Storage:* We must store the "Picklists" (List of Vendors, List of previous Chemicals, List of Contracts) in IndexedDB so the auto-complete works offline.
 - *Online:* Background Service Worker detects connection -> Uploads to Supabase Storage -> Gets URL -> Updates Transaction Record -> Clears local Base64.
3. **Commodity API (3rd Party):**
 - Use a serverless Edge Function (Supabase Functions) to proxy requests to an API like *Alpha Vantage* or *Commodities-API*.
 - *Why?* To hide API keys and cache results (reduce 3rd party costs).

5. UI/UX & Design Constraints

- **Touch Targets:** All interactive elements must have a min-height and min-width of **48px**.
- **Typography:** Base font size **16px** (inputs), headers **24px+**. High contrast (Black text on White/Light Gray background).
- **Input Types:**
 - Money fields must default to the numeric keypad.
 - Date fields must use the native OS date picker.
- **Feedback:**
 - Save actions must show a "Toast" notification: *"Saved to device"* (Offline) or *"Synced to Cloud"* (Online).
- **To prevent overwhelming the user with these new fields,** we will use Conditional Logic in the forms (however for debugging and testing purposes in the initial phase of the project there needs to be a toggle that can enable or disable conditional logic in the forms thus allowing the developer to see all the forms at once without clicking through them).
 - Default State: The "Add Record" screen shows only Amount, Date, and Category.
 - Condition 1 (Fertilizer): If Category = "Fertilizer", slide down inputs for Vendor, Product Name, N-P-K Calculator, Application Stage.
 - Condition 2 (Grain Sale): If Category = "Grain Sale", slide down inputs for Delivery Location, Contract ID, Moisture/Test Weight.
 - Condition 3 (Gov Payment): If Category = "Gov Payment", slide down Program Type selector.

6. Feature: The Commodity "Action" Dashboard

Objective: To give the user an overview of the current market conditions for their crops and give them more specific financial market related information.

A. The Tickers

- **Visual:** Rows of crop prices specific to which crops the user has. This would have typical information associated with each one like current price, % change, etc.

- **Content:**
 - **Market:** Chicago Board of Trade (CBOT) Futures for Corn (ZC), Soybeans (ZS), and Wheat (ZW) and others.
 - **Display:** Big bold price (e.g., "\$4.50"), Change indicator (Green Up Arrow / Red Down Arrow), and "cents change" (e.g., "+0.12").
 - **User Config:** User selects which crops appear here in Settings.

B. The "Local Basis" Widget (The Critical Differentiator)

- **Problem:** Farmers don't sell at the Board price; they sell at the **Cash Price** (Board + Local Basis).
- **Functionality:**
 - **Location Tracking:** The user can "Follow" specific local elevators (e.g., "ADM South", "Buhler Co-op").
 - **API Integration:** Fetch *Cash Bids* for these specific locations (via 3rd party API like Barchart or DTN).
 - **Visualization:** A simple leaderboard card:

Corn Cash Bids

- **ADM South:** \$4.35 (Basis: -0.15)
- **Buhler:** \$4.28 (Basis: -0.22)
- **Ethanol Plant:** \$4.45 (Basis: -0.05) *Highlighted as Best Price*

C. The "Profit Overlay" Chart (The Killer Feature)

- **Concept:** This charts the Market Price *against* the User's Real Data.
- **X-Axis:** Time (Next 6 months).
- **Y-Axis:** Price per Bushel.
- **Line 1 (Market):** The projected Futures price curve.
- **Line 2 (The "Red Line"):** The user's **Break-Even Price**.
 - *Calculation:* (Total Field Expenses / Estimated Yield).
 - *Dynamic:* As the user enters more expenses (fertilizer, fuel) or lowers yield estimates, this red line moves *up* in real-time.

- *Fixed:* After the harvest the user will enter the actual amount of yield, so now the breakeven line can remain at the price position where the field will breakeven (ie. If the user has 1000 bushels of X, and it costed them \$1000 overall, the breakeven indicator would be at \$1/bushel)
- **User Value:** Harry can look at the chart and instantly see: *"Corn is \$4.50, but my breakeven is \$4.60. I cannot sell today."*

D. Marketing Position Summary (Inventory Check)

- **Purpose:** To track how much grain is "Unsold" vs. "Contracted."
- **Data Source:** Aggregated from transactions (Harvest logs vs. Sales logs).
- **Visual:** A "Battery" style progress bar for each crop.
 - **Total Inventory:** 10,000 Bushels (Harvested).
 - **Sold/Forward Contracted:** 4,000 Bushels (40%).
 - **Unsold (Risk):** 6,000 Bushels (60%).
- **Action Button:** "Add Contract" button next to the Unsold amount to quickly log a forward sale.

6. Database Schema

6.1 Table: profiles (User Settings)

- **Purpose:** Extends Supabase auth.users with farm-specific preferences.
- **Columns:**
 - id (UUID, PK): References auth.users.id (On Delete Cascade).
 - farm_name (TEXT): Display name.
 - settings (JSONB): Stores user preferences to avoid extra tables.

- *Structure:*

```
JSON
{
  "default_currency": "USD",
```

```

    "measurement_system": "imperial",
    "dashboard_crops": ["Corn", "Soy"], // [cite: 109]
    "followed_elevators": ["ADM South", "Buhler"] //
  }

```

- created_at (TIMESTAMPTZ).

6.2 Table: fields (Geospatial Entity)

- **Purpose:** Represents physical land units for the map view.
- **Columns:**
 - id (UUID, PK): Default gen_random_uuid().
 - user_id (UUID, FK): References profiles.id.
 - name (TEXT): e.g., "North 40".
 - acres (NUMERIC): The manual "Official" size for financial calcs.
 - gis_acres (NUMERIC): Auto-calculated from polygon area.
 - boundary (GEOMETRY): PostGIS Polygon (SRID 4326).
 - created_at (TIMESTAMPTZ).

6.3 Table: seasons (Context & Planning Layer)

- **Purpose:** Links a Field to a Crop Year and handles Sharecropping & Breakeven targets.
- **Columns:**
 - id (UUID, PK).
 - field_id (UUID, FK): References fields.id.
 - year (INTEGER): e.g., 2025.
 - crop_type (TEXT): Enum e.g., 'Corn', 'Soy', 'Wheat'.
 - status (TEXT): 'Active', 'Archived'.
 - **Financial Logic (Sharecropping):**
 - landlord_share_percent (NUMERIC): Default 0.0. (e.g., 33.0 implies user keeps 67%) .

- landlord_name (TEXT): Optional.
- **Dashboard Logic (Breakeven):**
 - estimated_yield_per_acre (NUMERIC): User's target yield. *Used to draw the "Red Line" breakeven chart before harvest.*

6.4 Table: contracts (Marketing & Hedge Tracking)

- **Purpose:** Tracks "Sold/Forward Contracted" grain to power the Marketing Position "Battery" bar.
- **Columns:**
 - id (UUID, PK).
 - season_id (UUID, FK): References seasons.id.
 - contract_number (TEXT): e.g., "8K-6.27".
 - buyer_name (TEXT): e.g., "ADM South".
 - contract_type (TEXT): Enum 'Cash', 'Basis', 'Futures_Hedge'.
 - quantity_bushels (NUMERIC): Total bushels committed.
 - price_per_bushel (NUMERIC): Locked price.
 - delivery_deadline (DATE).
 - status (TEXT): 'Open', 'Fulfilled', 'Cancelled'.

6.5 Table: transactions (Master Ledger)

- **Purpose:** Single source of truth for all events (Income, Expense, Harvest).
- **Columns:**
 - id (UUID, PK).
 - season_id (UUID, FK): References seasons.id.
 - date (DATE).
 - type (TEXT): Enum 'INCOME', 'EXPENSE'.
 - category (TEXT): Enum 'Chemical', 'Fertilizer', 'Seed', 'Grain Sale', 'Insurance', 'Govt Payment', 'Fuel', 'Machine Hire', 'Tax', 'Interest', 'Other'.
 - vendor (TEXT): e.g., "MKC", "Buhler".

- amount (NUMERIC): Total Value (Negative for expense, Positive for income).
- quantity (NUMERIC): Amount of units (Bushels, Gallons).
- unit (TEXT): 'bu', 'gal', 'tons', 'lbs'.
- receipt_url (TEXT): Supabase Storage path.
- notes (TEXT).
- created_at (TIMESTAMPTZ).
- meta_data (JSONB): **Stores the conditional logic fields.**

JSONB Structures for meta_data:

- **CASE: category = 'Grain Sale' (Harvest/Income)**

```
JSON
{
  "contract_id": "uuid-from-contracts-table", //
  "delivery_location": "ADM South",          //
  "moisture": 14.5,    // % [cite: 51]
  "test_weight": 58.0, // Lbs/Bu [cite: 51]
  "protein": 11.2,    // % [cite: 51]
  "dry_bushels": 842.5 // Net Inventory calculated from moisture
}
```

- **CASE: category = 'Govt Payment' (Subsidies)**

```
JSON
{
  "program_type": "PLC", // Options: PLC, ARC, CFAP, ERP, MFP [cite: 55]
  "reference_year": 2024,
  "agency_ref": "USDA-XYZ"
}
```

- **CASE: category = 'Chemical' OR 'Fertilizer'**

JSON

```
{
  "product_name": "Liberty 280",    // [cite: 62]
  "application_stage": "Spring Weed", // Options: Pre-Plant, Starter, Top
  "weather_notes": "Windy, 15mph S", // [cite: 65]
  "nutrient_analysis": {           // [cite: 71]
    "n_actual": 80, // Calculated Lbs of Nitrogen [cite: 71]
    "p": 40,
    "k": 0,
    "s": 10
  }
}
```

6.6 Views (Reporting Layer)

View: view_field_pnl

- **Purpose:** Powers the "Global View" Leaderboard and "Profit Overlay".
- **Logic:**

1. Group transactions by season_id.
2. Sum amount where type = 'INCOME' as gross_revenue.
3. Sum amount where type = 'EXPENSE' as total_expenses.
4. Fetch landlord_share_percent from seasons.

5. **Calculated Net Income:**

$$\text{net_income} = (\text{gross_revenue} * ((100 - \text{landlord_share_percent})/100)) - \text{total_expenses}.$$

6. **Calculated Breakeven Price:**

$$\text{breakeven_price} = \text{total_expenses} / (\text{NULLIF}(\text{total_harvested_bushels}, 0)).$$

